

SOLID Principles — Cinema Ticket Booking System

SOLID Principle	Where Used in Cinema Ticket System	Simple Example	Purpose
S — Single Responsibility	BookingService , PriceCalculator , TicketPrinter	BookingService manages the booking process, PriceCalculator calculates the price, and TicketPrinter prints the ticket.	One class = one main job
O — Open/Closed	Payment hierarchy	Add a new NetBankingPayment class by extending Payment without changing existing payment classes.	Easy to extend without modifying existing code
L — Liskov Substitution	Payment → UpiPayment , CardPayment , CashPayment	If the system expects a Payment* , it can receive UpiPayment , CardPayment , or CashPayment and call pay() .	Child can safely replace parent
I — Interface Segregation	Payment interface	Payment only requires pay() . If refunds are added later, use a separate Refundable interface instead of forcing every payment type to implement refund() .	Avoid unnecessary methods
D — Dependency Inversion	BookingService → Payment	BookingService should receive a Payment& instead of creating CardPayment or UpiPayment itself.	Depend on abstractions, not concrete classes

Most Important Examples to Remember

Principle	Remember This
S	PriceCalculator only calculates
O	Add NetBankingPayment without changing old payment code
L	UpiPayment can replace Payment

Principle	Remember This
I	Don't force <code>CashPayment</code> to implement <code>refund()</code>
D	<code>BookingService</code> uses <code>Payment</code> , not <code>CardPayment</code> directly